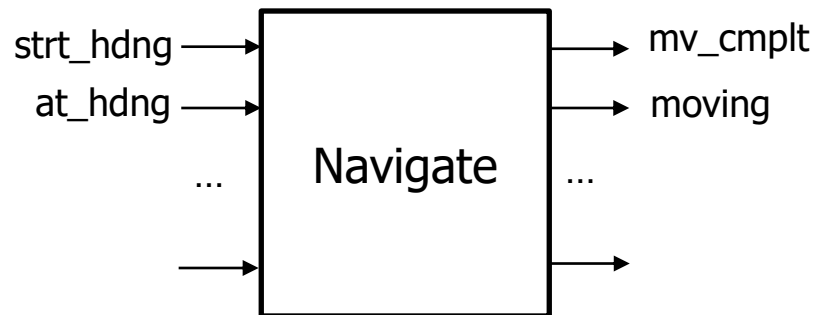




-





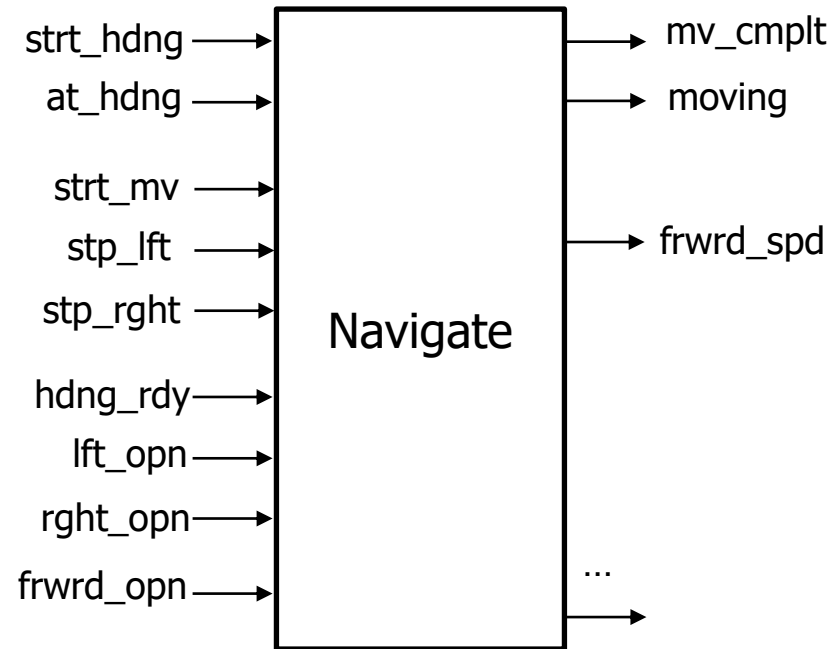
Exercise 13: Navigate Unit

Move command

- Get **strt_mv** cmd
- Accelerate **frwrд_spд** to MAX_FRWRD
 - Set initial speed by asserting **init_frwrд**
 - Assert **moving**
 - Ramp up frwrд_spд by asserting **inc_frwrд**
- On stopping condition, decelerate **frwrд_spд** to 0

Stopping Conditions

- We encounter a left opening (**stp_lft** cmd, lft_opn)
- We encounter a right opening (**stp_rght** cmd, rght_opn)
- We encounter wall in front ($\sim$ frwrд_opn)

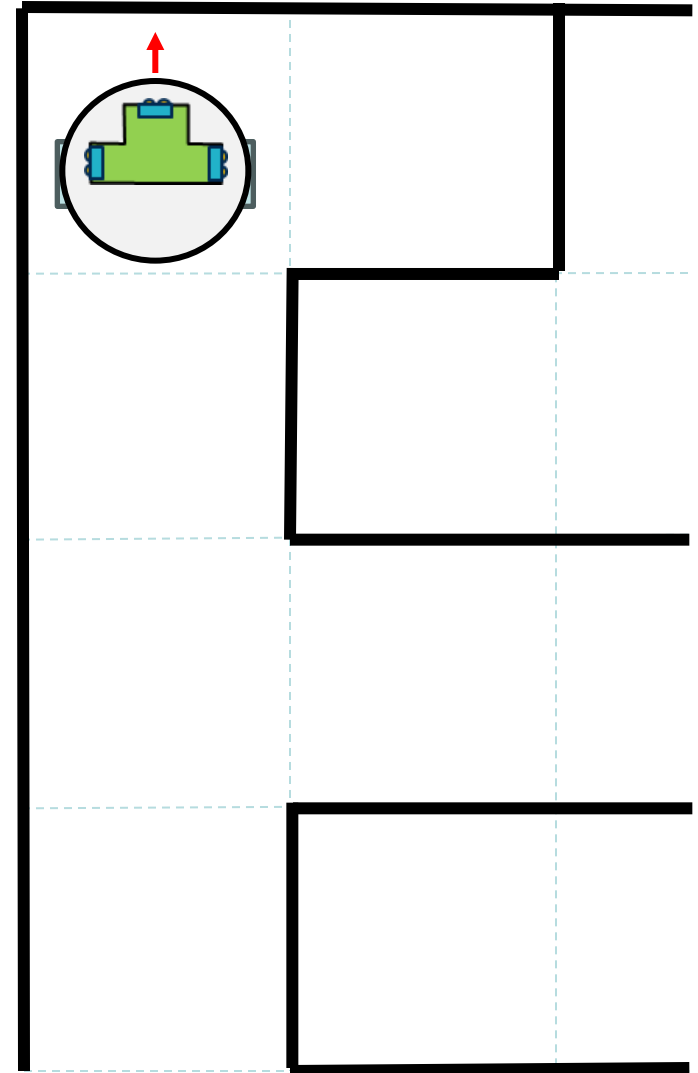




Exercise 13: Navigate Unit

We need 2 types of deceleration

- If we are at a wall, need to decelerate fast so we don't crash
→ decel. at 8x normal incr.

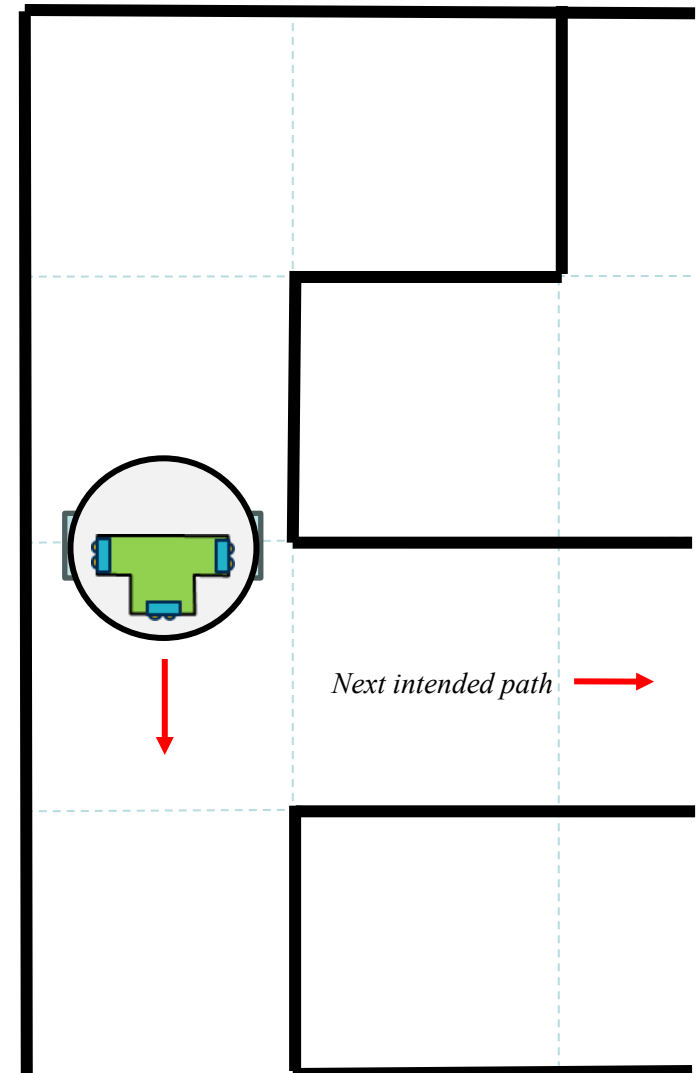




Exercise 13: Navigate Unit

We need 2 types of deceleration

- If we are at a wall, need to decelerate fast so we don't crash
→ decel. at 8x normal incr.
- If we are at a requested opening, need to decelerate so we travel another $\frac{1}{2}$ square
→ decel. at 2x normal incr.





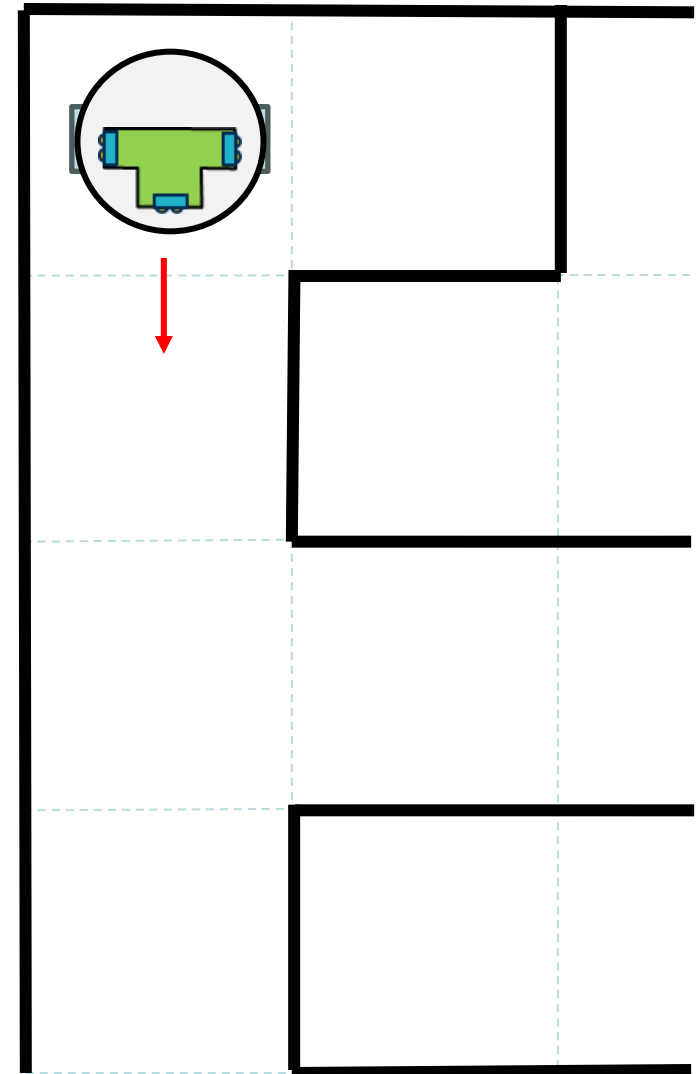
Exercise 13: Navigate Unit

We need 2 types of deceleration

- If we are at a wall, need to decelerate fast so we don't crash
→ decel. at 8x normal incr.
- If we are at a requested opening, need to decelerate so we travel another $\frac{1}{2}$ square
→ decel. at 2x normal incr.

Note:

In order to start moving when requested opening is already asserted (e.g., `lft_opn`), you need **rising edge detectors**
→ `lft_opn_rise` and `right_opn_rise`

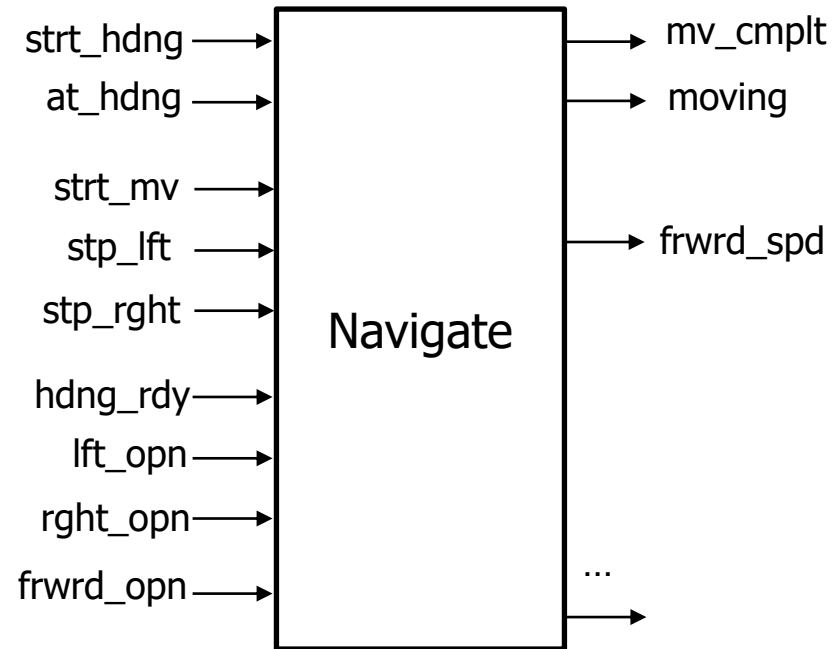




Exercise 13: Navigate Unit

Move command

- Get **strt_mv** cmd
- Accelerate **frwrд_spд** to MAX_FRWRD
 - Set initial speed by asserting **init_frwrд**
 - Assert **moving**
 - Ramp up frwrд_spд by asserting **inc_frwrд**
- On stopping condition, decelerate **frwrд_spд** to 0
 - **Fast decel**: ramp frwrд_spд down fast by asserting **dec_frwrд_fast**
 - **Normal decel**: ramp frwrд_spд down by asserting **dec_frwrд**
 - Assert **mv_cmplt**





Exercise 13: Navigate Unit

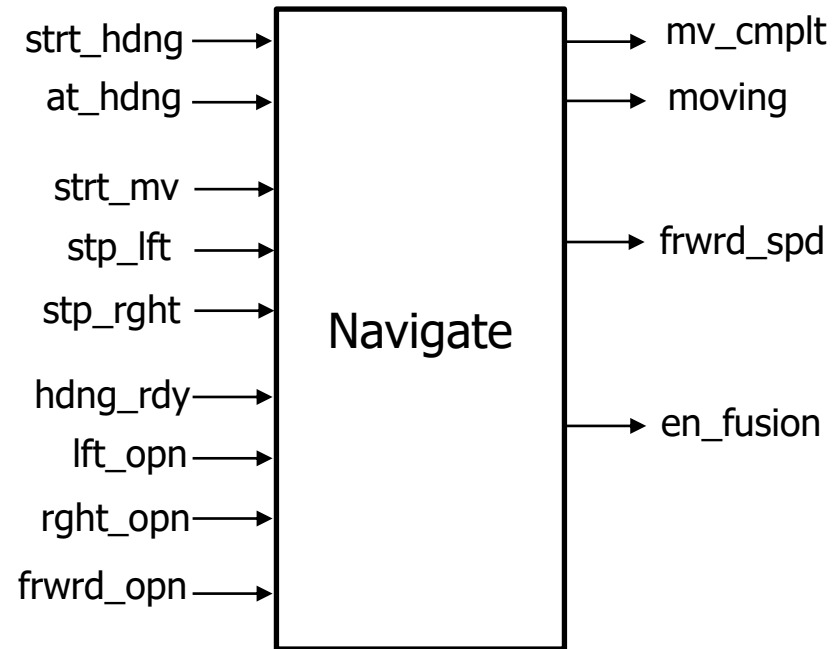
Generate **en_fusion**

- We want the IR sensors to affect navigation ONLY if it is moving forward
- Decode the current magnitude of frwrd_spd to assert en_fusion if it is greater than $\frac{1}{2}$ MAX_FRWRD

Implement parameter **FAST_SIM**

- Rate of incr. / decr. is controlled by magnitude of **frwrd_inc**
- Normal operation: 6'h02
- This will take long to simulate, use larger step during simulation
- Simulation: 6'h18

```
generate if (FAST_SIM) begin
    // set frwrd_inc to 6'h18
end else begin
    // set frwrd_inc to 6'h02
end
endgenerate
```





Exercise 13: Navigate Unit

- You are provided the **frwrд_spд** register implementation in **navigate_shell.sv** → Use this to create your **navigate.sv**
 - Add edge detectors on **lft_opn** and **rght_opn**
 - Implement frwrд_inc based on parameter FAST_SIM
 - Implement en_fusion signal
 - Implement the FSM
 - Understand requirements
 - Declare states and SM outputs
 - Implement state registers
 - Implement state transition and output logic
- A testbench with a few test cases is provided (**navigate_tb_starter.sv**)
- Complete it with more test cases & submit **navigate_tb.sv**
- Submit proof that it passed the self-checking testbench (transcript window)



Navigate Unit: FSM Functionality

- Sit in IDLE till told to turn to a new heading (**strt_hdng** asserted) or start a move (**strt_mv** asserted).
- If performing a heading change, simply assert **moving** and wait until **at_hdng** signal (*from **PID** unit*) is asserted. Assert **mv_cmplt** on way back to IDLE.
- If performing a move first initialize forward speed (**init_frwr**) and then ramp up to speed. In this state assert **moving** and **inc_frwr**.
 - Stay in the accelerate state because the **frwr_spd** register will saturate to **MAX_FRWRD** on its own. So effectively this is both accelerate and “cruise” state.
 - Leave the accelerate state when IR readings indicate stopping condition (*either obstacle in front ($\sim$ frwr_opn) or desired opening lft/right*)
- There are 2 decelerate states, they differ only in whether one is asserting **dec_frwr** or **dec_frwr_fast**. The SM stays in these state until the **frwr_spd** register is 0. Even though we are in the process of stopping in these states, **moving** should still be asserted. These states return to IDLE when completed (*and assert **mv_cmplt** on the way*)

SM output signals
moving
init_frwr
inc_frwr
dec_frwr
dec_frwr_fast
mv_cmplt